

دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر



یادگیری ماشین

تمرین شماره ۳

نام و نام خانوادگی
مرتضی ملکی نژاد شوشتری

شماره دانشجویی
۸۱۰۱۰۴۲۵۶

۲۷ بهمن ۱۴۰۴

۱	چکیده	۱
۲	Perceptron	۱
۲		۱.۱
۲		۲.۱
۳		۳.۱
۴		۴.۱
۴	 a	۱.۴.۱
۴	 b	۲.۴.۱
۵	 c	۳.۴.۱
۶	Representation Networks: Neural	۲
۷	 a۱	۱.۲
۷	 a۲	۲.۲
۷	 b۱	۳.۲
۷	 b۲	۴.۲
۷	 c۱	۵.۲
۷	 c۲	۶.۲
۹	Propagation Backward and Forward Networks: Neural	۳
۹		۱.۳
۹		۲.۳
۱۰		۳.۳
۱۰		۴.۳

۱۰		۵.۳
۱۰		۶.۳
۱۱		۷.۳
۱۱	Network Two-Layer a of Hessian	۴
۱۱		۱.۴
۱۱		۲.۴
۱۲		۳.۴
۱۲	LDA	۵
۱۳	 PCA	۱.۵
۱۴	Selection Feature	۶
۱۵	Classification Image Flower	۷

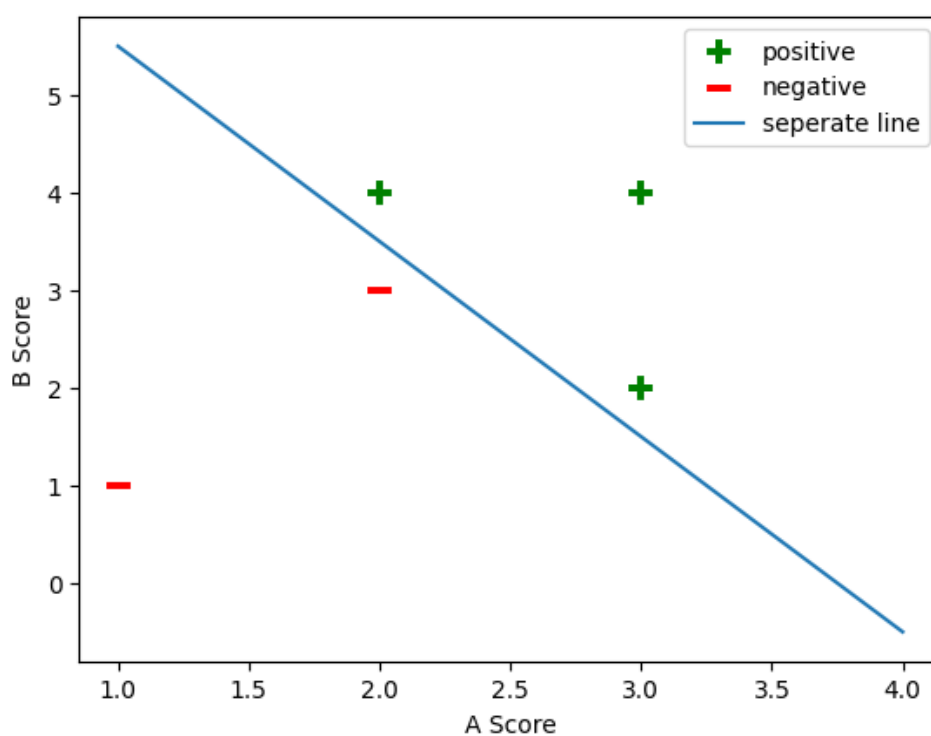
۱	نمودار نقاط باتوجه به امتیازها. داده‌ها بصورت خطی جداپذیر هستند، یکی
۲	از این خط‌ها رسم شده است.
۳	نمایش خط یادگرفته شده در مرحله ۲
۳	خروجی پرسپترون پس از ۵۰ بار یادگیری و با ضریب یادگیری 0.005. مدل
۴	موفق شده خطی را بیابد که دسته‌ها را بطور کامل جدا کرده است.
۴	هر تابع را چه مدل‌هایی می‌توانند یاد بگیرند.
۵	مقدار CCR بر اساس تعداد ویژگی با مدل Multinomial Naive Bayes
۶	مقدار ضرر در هر دور یادگیری برای مدل تک لایه
۷	مقدار ضرر در هر دور یادگیری برای مدل دو لایه
۸	مقدار ضرر در هر دور یادگیری برای مدل دو لایه همراه با یک سری بهبود
۱۶	برای جلوگیری از بیش‌برازش
۹	مقدار ضرر در هر دور یادگیری برای مدل دو لایه با استخراج ویژگی از alexnet
۱۰	مقدار ضرر در هر دور یادگیری برای مدل دو لایه با استخراج ویژگی از vgg16

فهرست جداول

۱	خروجی Perceptron در هر گام	۳
۲	ابعاد هر پارامتر در حالت معمولی	۹
۳	ابعاد هر پارامتر در حالت vectorize	۹
۴	دقت مدل به ازای ۷۵ ویژگی	۱۴
۵	ویژگی‌های انتخاب شده در هر دو روش برای داده tinyMNIST در سوال ۷	۱۸
۶	دقت هر مدل	۱۹

Perceptron

۱.۱



شکل ۱: نمودار نقاط با توجه به امتیازها. داده‌ها بصورت خطی جداپذیر هستند، یکی از این خط‌ها رسم شده است.

۲.۱

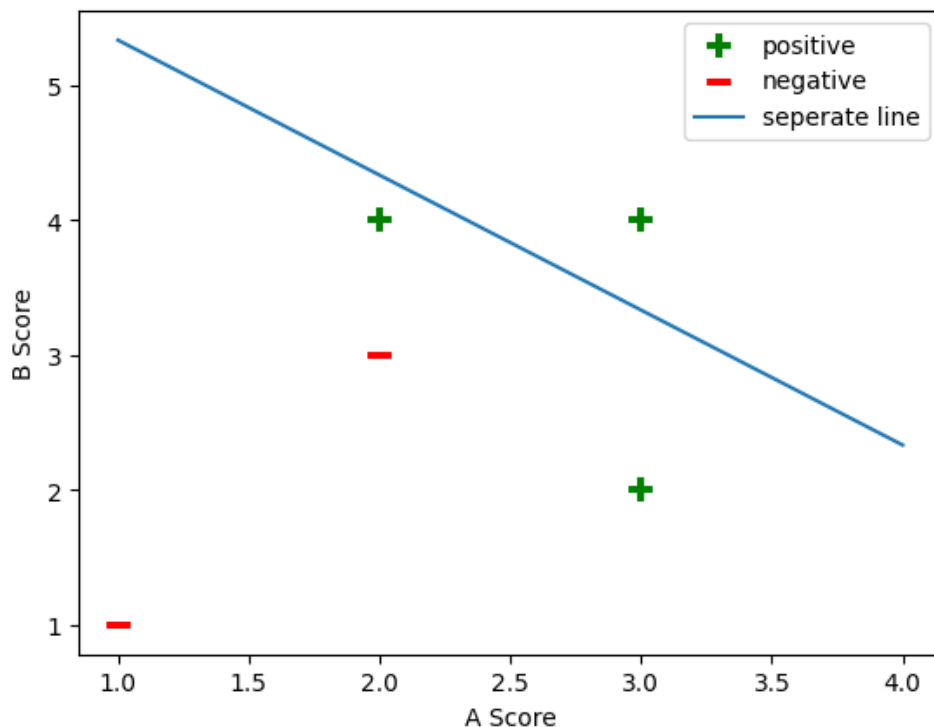
مقدار learning rate برابر 0.05 در نظر گرفته شده است.

جدول ۱: خروجی Perceptron در هر گام

Step	Weight	Score	Correct?
1	[-1, 0, 0]	$-1.1 + 0.1 + 0.1 = -1$	yes
2	[-1, 0, 0]	-1	no
3	[-0.95, 0.15, 0.1]	-0.25	no
4	[-0.9 0.25 0.3]	1.05	yes
5	[-0.9 0.25 0.3]	0.5	no

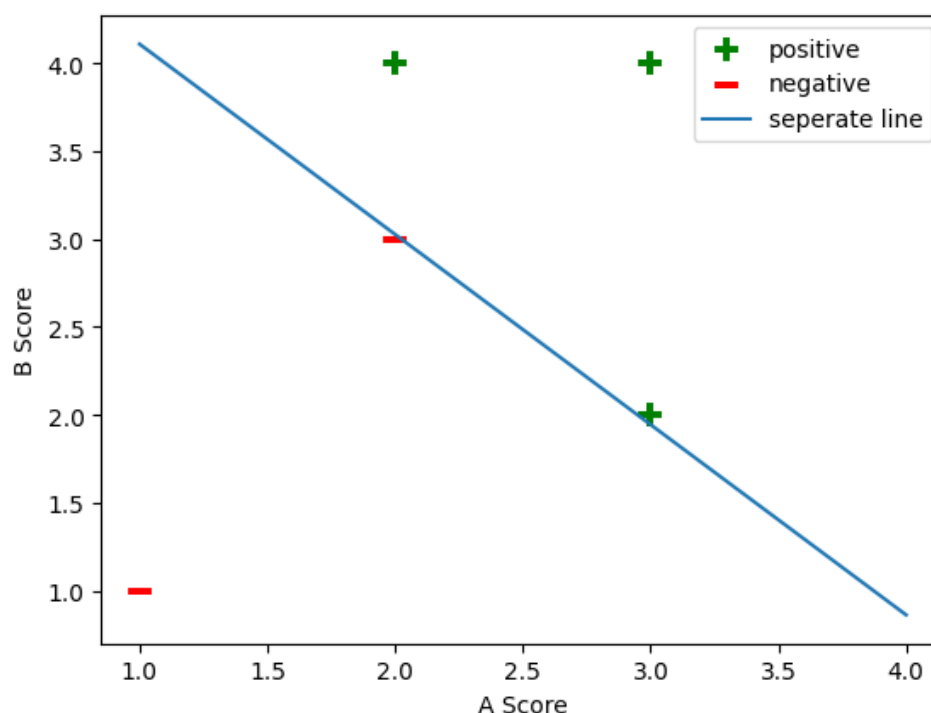
۳.۱

باتوجه به نمودار کشیده شده، هنوز خط جداسازی به طور کامل یادگرفته نشده است.



شکل ۲: نمایش خط یادگرفته شده در مرحله ۲

البته اگر همین پرسپترون به دفعات بیشتری آموزش ببیند می تواند این داده را بصورت کامل جداسازی کند.



شکل ۳: خروجی پرسپترون پس از ۵۰ بار یادگیری و با ضریب یادگیری ۰.۰۰۵. مدل موفق شده خطی را بیابد که دسته‌ها را بطور کامل جدا کرده است.

۴.۱

در هر مورد باید بررسی کرد که دسته‌ها بصورت خطی جداپذیر هستند یا خیر. یعنی آیا وزنی هست که مدل با رسیدن به آن‌ها بتواند با علامت score دسته‌ها را پیش‌بینی بکند یا خیر. اگر وجود داشته باشد یعنی مدل می‌تواند (و ابر متغیرها باید درست تنظیم شوند) و اگر نباشد مدل با هیچ وزنی نمی‌تواند دسته‌ها را جدا کند.

1.4.1 a

بله. درواقع مدل باید بتواند به وزن $[-8, 1, 1]$ برسد.

1.4.2 b

این داده بصورت خطی جداپذیر نیست (درواقع ناحیه بین دو خط در صفحه است) و مدل نمی‌تواند این را بصورت کامل یاد بگیرد.

1.4.3 c

این مورد نیز خطی جداپذیر نیست (در واقع خود یک خط است) و مدل نمی‌تواند این را یاد بگیرد.

Neural Networks: Representation

ابتدا هر شبکه بررسی می‌شود:

- **G1** این شبکه چون عنصر غیر خطی ندارد فقط قادر است توابع خطی را پیش‌بینی کند. همچنین چون عنصر بایاس ندارد، خط باید حتماً از مبدا بگذرد.
 - **G2** مشابه قسمت قبل این شبکه نیز عنصر غیر خطی ندارد و نمی‌تواند توابع غیرخطی را پیش‌بینی کند. ولی چون بایاس دارد، لزومی ندارد خط از مبدا بگذرد.
 - **G3** با اینکه به این شبکه عنصر غیر خطی relu اضافه شده است، ولی relu بدون بایاس و با تنها یک بایاس ضعیف است. در اینجا عملاً وزن‌های گره دوم تاثیری ندارند (تنها این قابلیت را به مدل می‌دهند که خروجی منفی بدهد).
 - **G4** خروجی این قسمت بعد از اعمال relu یک خط شکسته شبیه به خود تابع relu است با این تفاوت که نقطه شکست روی صفر نیست و با مقدار w_1 و b_1 تعریف می‌شود. گره دوم ولی تاثیر زیادی ندارد و صرفاً شیب خط را (در جایی که خود تابع بخاطر اعمال relu صفر نشده) عوض می‌کند. یعنی عملاً قابلیت جدیدی نمی‌دهد (مشابه قسمت ۳ تنها فایده این گره این است که تابع می‌تواند مقادیر منفی را نیز خروجی دهد).
 - **G5** این تابع بسیار مشابه قسمت ۴ است و تنها یک بایاس به آن اضافه شده است. یعنی می‌تواند هر خط شکسته‌ای که یک قسمت آن موازی محور x ها هست را پیش‌بینی کند.
- شبکه‌های دوبعدی عملاً ترکیب دوتا از این شبکه‌های تک بعدی هستند. در موارد غیر خطی قاعده مهم این است که اگر نقاط شکست از تعداد توابعی که relu روی آن‌ها اعمال شده است بیشتر باشند نمی‌توان آن‌ها را پیش‌بینی کرد.

2.1 a1

این یک تابع خطی گذرنده از مبدا است و همه شبکه‌ها می‌توانند آن را پیش‌بینی کنند.

2.2 a2

این یک تابع خطی است که از مبدا نمی‌گذرد و به غیر از $G1$ و $H1$ سایر شبکه‌ها می‌توانند آن را پیش‌بینی کنند.

2.3 b1

این یک تابع شکسته است که هر دو تکه آن شیب غیر صفر دارند. برای همین از شبکه‌های یک بعدی هیچ کدام نمی‌توانند آن را پیش‌بینی کنند ولی در شبکه‌های دو بعدی، هر مسیر می‌تواند یک تکه را یاد بگیرد (و تکه دیگر صفر باشد) و سپس این دو لایه با هم جمع می‌شوند چون نقطه شکست روی مبدا است نیازی به بایاس نیست پس یعنی شبکه‌های $H4$, $H3$ و $H5$ می‌توانند این تابع را یاد بگیرند.

2.4 b2

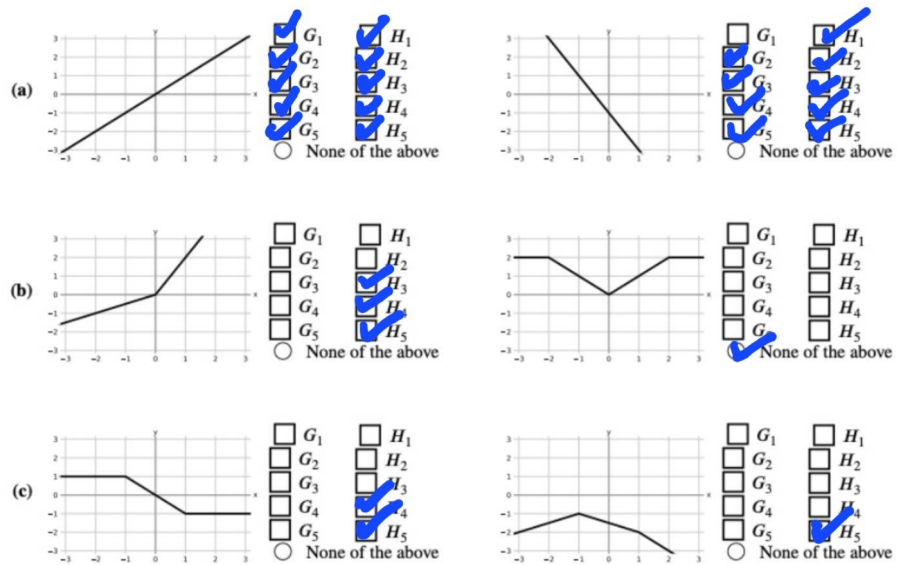
خروجی relu از یک طرف شیب غیر صفر دارد ولی این شکل از هر دو طرف شیب صفر دارد. برای خروجی گرفتن همچنین شکلی حداقل به دو relu متوالی نیاز است. پس هیچ یک از شبکه‌ها نمی‌توانند این تابع را یاد بگیرند. تعداد نقاط شکست برابر ۳ است یعنی شبکه‌های دوبعدی نیز نمی‌توانند بطور دقیق همه تابع را بسازند.

2.5 c1

با هیچ یک از شبکه‌های یک بعدی نمی‌توان ولی با شبکه دو بعدی می‌توان صرفاً باید یکی از قسمت‌هایی که شیب صفر دارند را با خنثی کردن ابعاد خروجی گرفت. پس یعنی نیاز به شبکه‌هایی داریم که پس از یک لایه relu بتوانند خروجی‌های تابع شکسته با bias بدهند. یعنی شبکه‌های $H4$ و $H5$.

2.6 c2

تابع دو نقطه شکست دارد. نکته مهم این است که با این که این تابع عملاً ترکیب سه خط است ولی شیب قسمت وسط برابر جمع شیب دوتکه دیگر است. پس یعنی می‌توان با دو تابع شکسته آن را ساخت. همچنین چون تابع شیفت یافته است باید در لایه پس از relu بایاس داشت پس یعنی فقط مدل $H5$ می‌تواند این تابع را بسازد.



شکل ۴: هر تابع را چه مدل‌هایی می‌توانند یاد بگیرند.

Neural Networks: Forward and Backward Propagation

۱.۳

جدول ۲: ابعاد هر پارامتر در حالت معمولی

Name	Dimension
W_1	$D_{a_1} \times D_x$
b_1	$D_{a_1} \times 1$
W_2	$1 \times D_{a_1}$
b_2	1×1

در حالت vectorize فرمول قسمت z به این صورت در می آید: $z = XW + b$

جدول ۳: ابعاد هر پارامتر در حالت vectorize

Name	Dimension
X	$n \times D_x$
Y	$n \times 1$
W_1	$D_x \times D_{a_1}$
b_1	$n \times D_{a_1}$
W_2	$D_{a_1} \times 1$
b_2	$n \times 1$

۲.۳

چون نمونه‌ها از هم مستقل هستند عملاً باید مقدار مشتق تابع $\frac{-1}{m}L^{(i)}$ را نسبت به $\hat{y}^{(i)}$ حساب کرد. پس داریم:

$$\begin{aligned}\frac{\partial J}{\partial \hat{y}^{(i)}} &= \frac{-1}{m} * \frac{\partial(y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)}))}{\partial \hat{y}^{(i)}} \\ &= \frac{-1}{m} * \left(\frac{y^{(i)}}{\hat{y}^{(i)}} + \frac{(y^{(i)} - 1)}{1 - \hat{y}^{(i)}} \right)\end{aligned}$$

برای همه نمونه‌ها این مقدار جمع می‌شود پس داریم:

$$\frac{\partial J}{\partial \hat{y}} = \frac{1}{m} \sum_{i=1}^m \left(\frac{1 - y^{(i)}}{1 - \hat{y}^{(i)}} - \frac{y^{(i)}}{\hat{y}^{(i)}} \right)$$

در کسرهای اگر حالت صفر صفرم رخ داد، جواب صفر است زیرا در صورت صفر مطلق رخ می‌دهد و در مخرج صفر حادی.

۳.۳

برابر با مشتق تابع سیگنویید بر حسب z می‌شود.

$$\frac{\partial \hat{y}}{\partial z_2^{(i)}} = \frac{e^{-z_2^{(i)}}}{(1 + e^{-z_2^{(i)}})^2}$$

۴.۳

z_2 یک تابع خطی بر حسب a_1 است که وزن آن W_2 است. پس یعنی مقدار $\delta_3^{(i)}$ برابر با W_2 است.

۵.۳

مشتق تابع relu یک تابع بصورت زیر است:

$$U(x) = \begin{cases} 0 & (x < 0) \\ 1 & (x \geq 0) \end{cases}$$

پس با توجه به اینکه $a_1 = \text{relu}(z_1)$ مقدار $\frac{\partial a_1}{\partial z_1}$ برابر با $U(z_1)$ می‌شود.

۶.۳

داریم:

$$z_1 = W_1 x^{(i)} + b_1 \Rightarrow \frac{\partial z_1}{\partial W_1} = x^{(i)}$$

۷.۳

داریم:

$$\begin{aligned}\frac{\partial J}{\partial W_1} &= \frac{\partial J}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial z_2} * \frac{\partial z_2}{\partial a_1} * \frac{\partial a_1}{\partial z_1} * \frac{\partial z_1}{\partial W_1} \\ &= \frac{-1}{m} \sum_{i=1}^m \left(\left(\frac{1 - y^{(i)}}{1 - \hat{y}^{(i)}} - \frac{y^{(i)}}{\hat{y}^{(i)}} \right) * \left(\frac{e^{-z_2^{(i)}}}{(1 + e^{-z_2^{(i)}})^2} \right) * (W_2) * U(z_1) * x^{(i)} \right)\end{aligned}$$

Hessian of a Two-Layer Network

۱.۴

داریم:

$$\begin{aligned}\frac{\partial E_n}{\partial w^{(2)}} &= \frac{\partial E_n}{\partial a_k} * \frac{\partial a_k}{\partial w^{(2)}} = \delta_k z_j \\ \Rightarrow \frac{\partial^2 E_n}{\partial w_j^{(2)} \partial w_{j'}^{(2)}} &= \frac{\partial(\delta_k z_j')}{\partial w_{j'}^{(2)}} = z_j \frac{\partial \delta_k}{\partial w_{j'}^{(2)}} = z_j * \frac{\partial \delta_k}{\partial a_k} * \frac{\partial a_k}{\partial w_{j'}^{(2)}} \\ &= z_j * M_{jj'} * z_{j'} = z_j z_{j'} M_{jj'}\end{aligned}$$

۲.۴

داریم:

$$\frac{\partial E_n}{\partial w_j^{(1)}} = \frac{\partial E_n}{\partial a_k} * \frac{\partial a_k}{\partial z_j} * \frac{\partial z_j}{\partial w_j^{(1)}} = \delta_k * w^{(2)} * h'(a_j) * x$$

$$\begin{aligned}
\Rightarrow \frac{\partial^2 E_n}{\partial w_j^{(1)} \partial w_{j'}^{(1)}} &= \frac{\partial(\delta_k * w^{(2)} * h'(a_j) * x_j)}{\partial w_{j'}^{(1)}} = w^{(2)} * x_j * \frac{\partial \delta_k * h'(a_j)}{\partial w_{j'}^{(1)}} \\
&= w^{(2)} * x_j * \left(\frac{\partial \delta_k}{\partial a_k} * \frac{\partial a_k}{z_j} * \frac{\partial z_j}{w_{j'}^{(1)}} * h'(a_j) + \delta_k * \frac{\partial h'(a_j)}{\partial a_j} * \frac{\partial a_j}{w_{j'}^{(1)}} \right) \\
&= w^{(2)} * x_j (M_{jj'} * w^{(2)} * h'(a_{j'}) * x_{j'} + \delta_k * h''(a_j) * 0) \\
&= w^{(2)^2} * x_j * x_{j'} * M_{jj'} * h'(a_j) * h'(a_{j'})
\end{aligned}$$

منظور از $w^{(2)}$ مقدار جمع کل وزن‌های لایه دوم می‌باشد.

۳.۴

داریم:

$$\frac{\partial E_n}{w_j^{(2)} w_i^{(1)}} = \frac{\partial(\delta_k z_j)}{\partial w_i^{(1)}} = \frac{z_j \partial(\delta_k)}{\partial w_i^{(1)}} + \frac{\delta_k \partial(z_j)}{\partial w_i^{(1)}} = z_j M_{ij} w^{(2)} h'(a_j) x_i + \delta_k h'(a_j) x_i$$

LDA

اگر در عبارت $\sum_{j=1}^{n_2} (y_i - y_j)^2$ مقدار y_i برابر با میانگین باشد، حاصل جمع برابر با همان s_2 می‌شود. حال اگر در نظر بگیریم که $y_i = m_2 + \delta_i$ داریم:

$$\begin{aligned}
\sum_{j=1}^{n_2} (y_i - y_j)^2 &= \sum_{j=1}^{n_2} (y_j - m_2 - \delta_i)^2 = \sum_{j=1}^{n_2} ((y_j - m_2)^2 + \delta_i^2 - 2\delta_i(y_j - m_2)) \\
&= s_2^2 + n_2 \delta_i^2 + 0 = s_2^2 + n_2 \delta_i^2
\end{aligned}$$

(چون حاصل جمع تفاوت y_2 از m_2 صفر می‌شود، عبارت سوم صفر شد.)

پس مقدار J برابر است با:

$$J = \frac{1}{n_1 n_2} \sum_{i=1}^{n_1} \sum_{j=1}^{n_2} (y_1 - y_2)^2 = \frac{1}{n_1 n_2} \sum_{i=1}^{n_1} (s_2^2 + n_2 \delta_i^2)$$

$$\begin{aligned}
&= \frac{1}{n_1 n_2} \sum_{i=1}^{n_1} (s_2^2 + n_2 (y_i - m_2)^2) = \frac{s_2^2}{n_2} + \frac{1}{n_1} \sum_{i=1}^{n_1} (y_i - m_2)^2 \\
&= \frac{s_2^2}{n_2} + \frac{1}{n_1} \sum_{i=1}^{n_1} (y_i - m_1 + m_1 - m_2)^2 \\
&= \frac{s_2^2}{n_2} + \frac{1}{n_1} \sum_{i=1}^{n_1} ((y_i - m_1)^2 + (m_1 - m_2)^2 + 2(y_i - m_1)(m_1 - m_2)) \\
&= \frac{s_2^2}{n_2} + \frac{s_1^2}{n_1} + (m_1 - m_2)^2
\end{aligned}$$

که این همان عبارت صورت سوال است.

5.1 PCA

اگر با یک تبدیل خطی بخواهیم داده‌ها بیشترین پراکندگی را از هم داشته باشند داریم:

$$y_i = w^T x_i \quad w^T w = I \quad v = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$$

چون x ها میانگین صفر دارند، میانگین y نیز صفر است (زیرا ترکیب خطی است). پس

$$v = \frac{1}{n} \sum_{i=1}^n (y_i)^2 \quad \text{یعنی هدف بیشینه کردن مقدار } v \text{ است. پس داریم:}$$

$$v = \sum_{i=1}^n (y_i)^2 = \sum_{i=1}^n (w^T x_i)^2 = \sum_{i=1}^n (w^T x_i)(x_i^T w) = \frac{1}{n} w^T \left(\sum_{i=1}^n x_i x_i^T \right) w = \frac{1}{n} w^T S w = w^T S w$$

(چون میانگین صفر است، ماتریس کواریانس همان جمع ضرب ها شد)

حال نیاز است تا مقداری برای w انتخاب شود تا این عبارت را بیشینه کند (اگر شرط اندازه

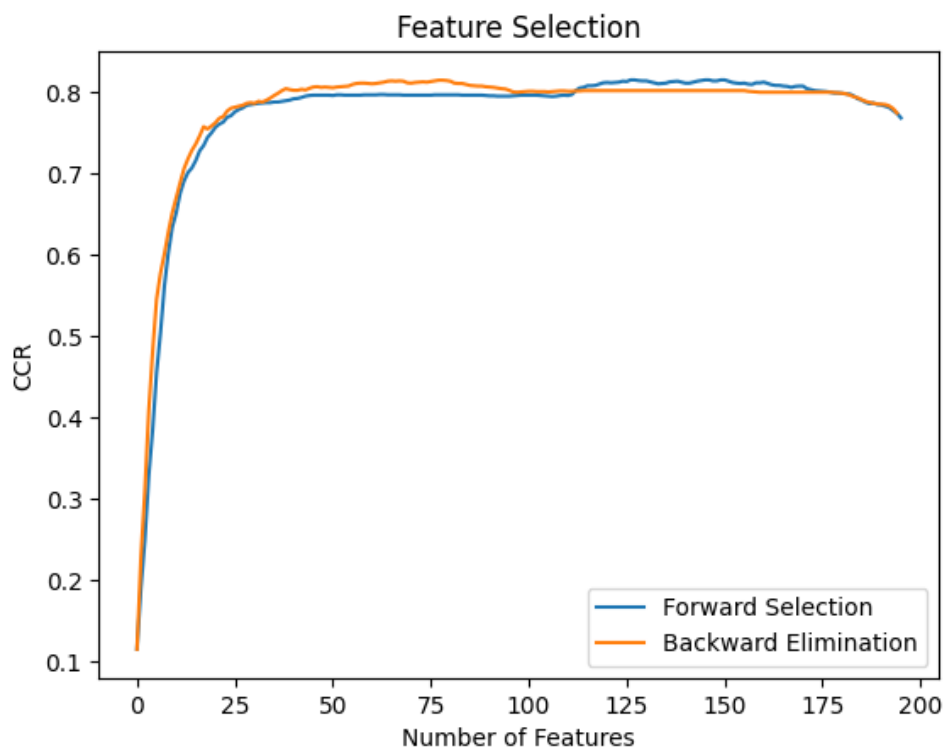
برای وزن‌ها نباشد، صرفاً می‌توان وزن‌های بسیار بزرگ انتخاب کرد و به واریانس‌های زیاد رسید).

اگر از شیوه لاگرانژین استفاده کنیم داریم:

$$L = w^T S w - \lambda (w w^T - 1) \xrightarrow{\text{derivative to } w} \frac{\partial L}{\partial w} = 2S w - 2\lambda w = 0 \Rightarrow S w = \lambda w$$

Feature Selection

با پیاده‌سازی هر دو روش و رسم نمودار خطی می‌توان دید بعد از انتخاب حدود هفتاد و پنج ویژگی دیگر مقدار CCR خیلی تغییری نمی‌کند.



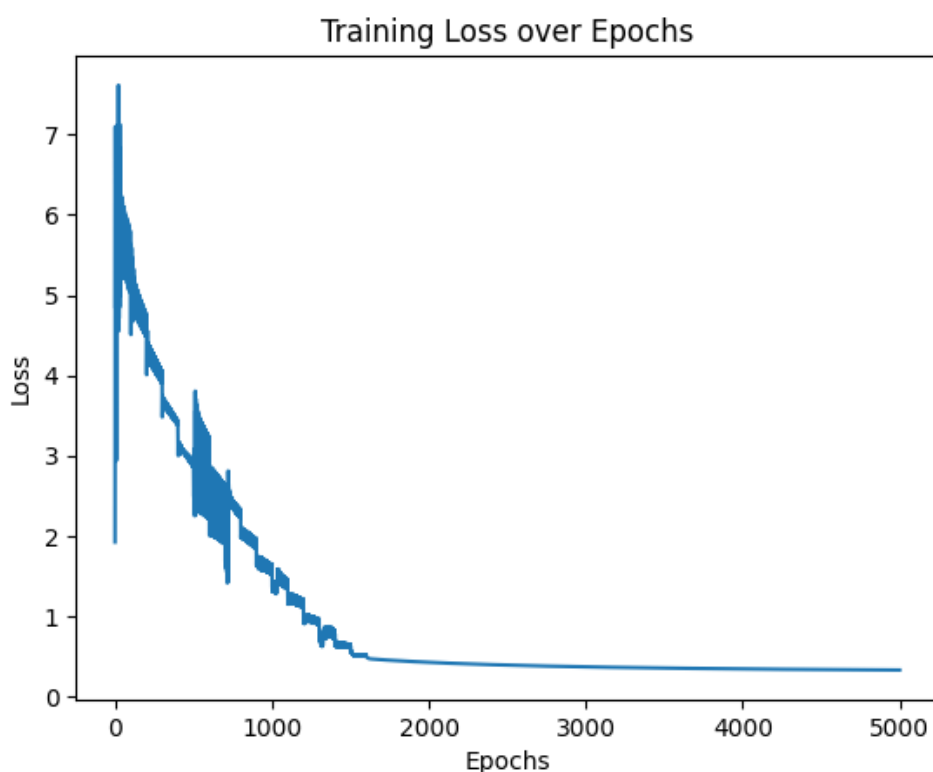
شکل ۵: مقدار CCR بر اساس تعداد ویژگی با مدل Multinomial Naive Bayes

جدول ۴: دقت مدل به‌ازای ۷۵ ویژگی

Feature selection method	CCR
Forward Selection	0.7964
Backward Elimination	0.8124

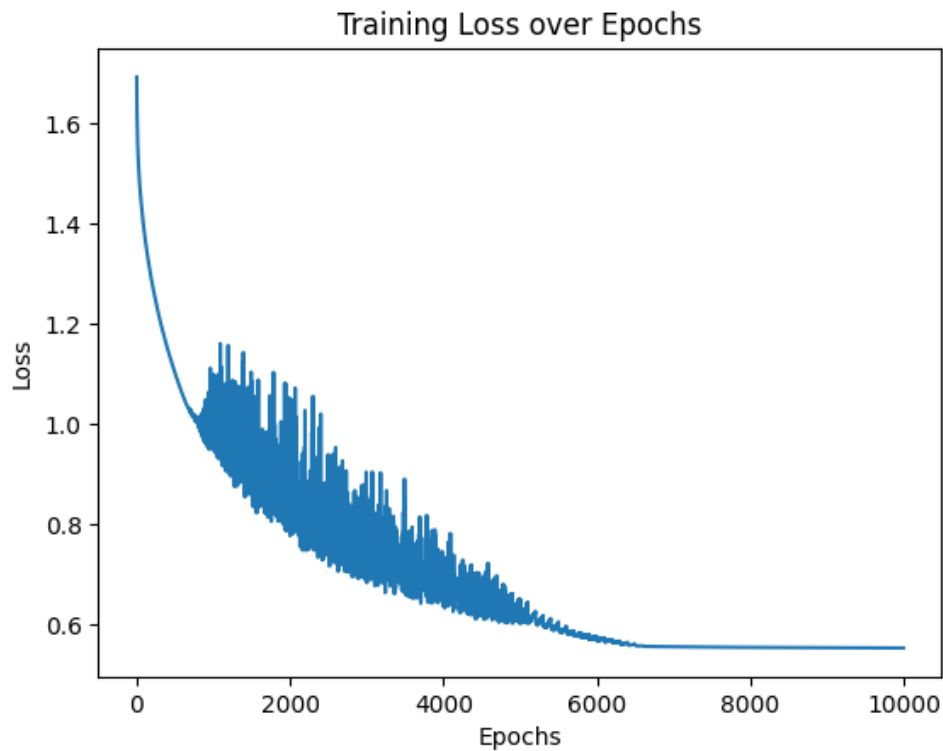
Flower Image Classification

با یک شبکه بدون لایه پنهان، اگر برای تابع خطا از cross entropy استفاده شود و از scale کردن ورودی و learning rate decay استفاده شود و برای مقدار اولیه وزن‌ها از روش Xavier استفاده شود، به ازای ضریب یادگیری اولیه 0.005 و ۳۰۰۰ دور یادگیری، می‌توان به دقت ۹۵ درصد روی داده‌های train و ۶۴ درصد روی داده‌های validation رسید. البته همچنان بعد از این تعداد دور هم دقت مدل هم روی داده‌های train و هم روی validation به سمت بهبود می‌رود ولی مقدار بهبود ناچیز است.

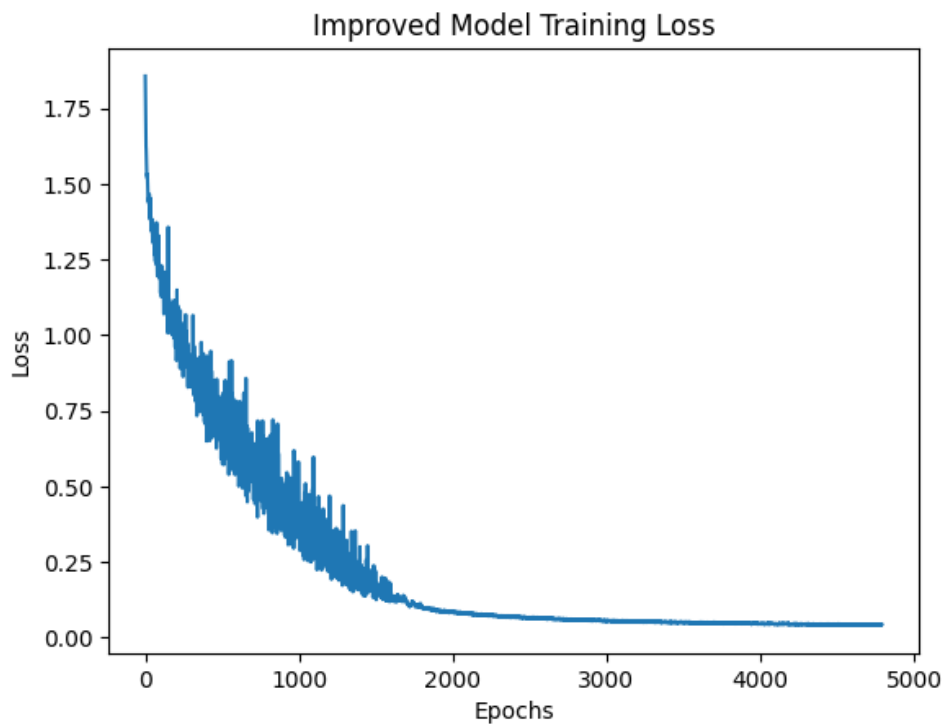


شکل ۶: مقدار ضرر در هر دور یادگیری برای مدل تک لایه چون بین مجموعه داده train و test فاصله زیاد است، انتظار می‌رود استفاده از روش‌هایی مثل regularization به دقت روی validation کمک کند و با استفاده از تکنیک‌های ذکر شده می‌توان به دقت 0.9975 روی داده train و 0.72 روی داده validation رسید.

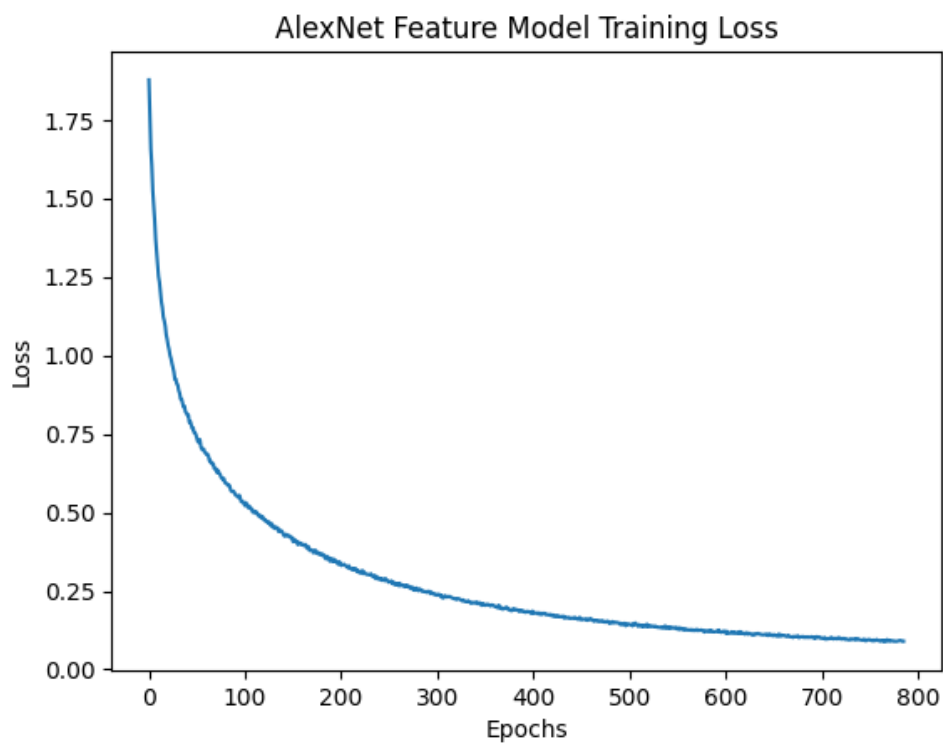
با استفاده از transfer learning و آموزش شبکه روی ویژگی‌های استخراج شده توسط شبکه alexnet و یا vgg16 می‌توان به دقت 0.9943 روی داده train و 0.8823 روی validation رسید.



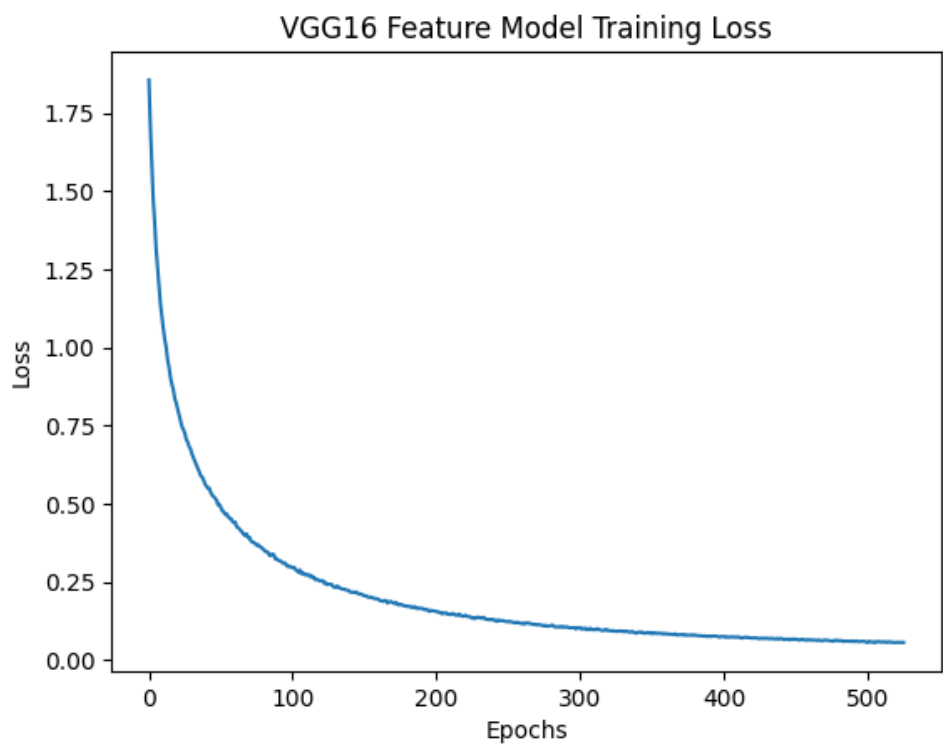
شکل ۷: مقدار ضرر در هر دور یادگیری برای مدل دو لایه



شکل ۸: مقدار ضرر در هر دور یادگیری برای مدل دو لایه همراه با یک سری بهبود برای جلوگیری از بیش‌برازش



شکل ۹: مقدار ضرر در هر دور یادگیری برای مدل دو لایه با استخراج ویژگی از alexnet



شکل ۱۰: مقدار ضرر در هر دور یادگیری برای مدل دو لایه با استخراج ویژگی از vgg16

جدول ۵: ویژگی‌های انتخاب شده در هر دو روش برای داده tinyMNIST در سوال ۷

Forward selection	Backward elimination
0-19	20
21	23
23-29	33
33	35-37
36	45-46
41-42	48-50
44	52-53
48-49	58
52	60-66
54-55	68
58	73-79
63	87-93
69	102-109
74-75	116
78-79	118-120
83	123
90-91	130-134
96-97	136
99	138
102	142-143
104	145-151
106	156-157
109	161-162
113	164
119	172-174
121	186
124-125	189
131	
138	
141	
144-145	
152	
160	
164	
166	
171-172	
178-180	
193	

جدول ۶: دقت هر مدل

Model	Train Loss	Train Accuracy	Validation Accuracy
one layer	0.3679	0.9316	0.622
two layer	0.4042	0.8947	0.6449
two layer improved	0.0443	0.9975	0.7201
alexnet + two layer improved	0.1	0.9943	0.8823
vgg16 + two layer improved	0.0566	0.9983	0.8823